CSE 291A Phase 1: Where Time and Energy Go in a Computer-Use Agent on CPU vs. GPU

[Lisa Wang Liw069@ucsd.edu]

ABSTRACT A computer-use agent runs two models at every step: a planner that decides the next action, and a grounding model that turns the action and a screenshot into click coordinates. In Phase 1 we built a measurement harness for this two-stage pipeline and used it to study where latency, memory and GPU energy go when each stage runs on CPU or GPU. We use Qwen2.5-VL-3B for grounding and Qwen2.5-3B for planning, both 4-bit quantized and served with llama.cpp, and evaluate on ScreenSpot with a fixed dev/eval split. On CPU, almost all time is spent processing the input (prefill). On an NVIDIA L4, one pipeline step drops from 56.1 s to 1.11 s on the same machine. Moving only grounding to the GPU helps little, because the CPU planner becomes the bottleneck while the GPU sits idle. These results motivate an input-adaptive policy for Phase 2.

1. INTRODUCTION

Edge computer-use agents should run on local hardware, so the cost of each agent step matters as much as accuracy. Most GUI grounding work reports accuracy only, and most inference benchmarks look at a single chat model. We look at the full agent step instead. Our questions are: (1) where does time go in each stage, (2) how does it change with input size and device, and (3) which placement of the two stages on CPU and GPU works best. In Phase 1 we provide a reproducible workload and evaluation procedure, initial measurements on CPU and GPU, and a Phase 2 direction based on these measurements.

2. EXPERIMENT SETUP

2.1. Pipeline and placements

Each stage runs in its own `llama-server` process (llama.cpp [2]) with an OpenAI-compatible API: grounding on port 8001 and the planner on port 8002. A server runs either fully on the GPU (`-ngl 99`) or fully on the CPU, where we also hide the GPU with `CUDA_VISIBLE_DEVICES=""`. We compare three placements: **B1** all CPU, **B2** grounding on GPU and planner on CPU, and **B3** all GPU.

2.2. Models and workload

Grounding uses Qwen2.5-VL-3B-Instruct [3] (Q4_K_M) with a Q8_0 vision projector. The planner uses Qwen2.5-3B-Instruct (Q4_K_M). We chose 3B models so that CPU runs finish in reasonable time. All model files are checked by SHA-256 and are identical on every machine.

For grounding we use ScreenSpot [1] (1,272 screenshots from web, mobile and desktop, each with an instruction and a target box). We split it once into 381 dev and 891 eval samples. All tuning and measurement use dev, and eval is used only for one final check. A prediction is correct if the click falls inside the target box; the strict score also rejects box-shaped outputs. Screenshots are resized to a pixel budget of 125k, 250k or 500k pixels. The planner gets synthetic contexts of 500, 2,000 or 4,000 tokens and generates up to 48 tokens. In the pipeline, grounding receives the ground-truth instruction, so that cost is not mixed with planner quality.

2.3. Harness and protocol

Our harness (`ecpipe`) sends streaming requests and records time to first token (TTFT, close to prefill time), decode speed, total latency and failures. A monitor samples process memory, GPU memory and GPU power (NVML), and we integrate power to get energy per request. Each run saves all settings, versions and model names in `meta.json` and every request in `records.jsonl`. We turn the prompt cache off, use one server slot, report the first request separately from warm requests, and repeat each run (grounding 100×3, planner 10–30×3, pipeline 20×2). All runs reported here had zero failures.

2.4. Hardware

We use two machines. UCSD DataHub has 8 vCPUs and llama.cpp build 11889. AWS `g6.2xlarge` has one NVIDIA L4 (23 GB), 8 vCPUs (4 physical cores), CUDA 13.2 and llama.cpp build 12020. All CPU-vs-GPU comparisons use the AWS machine only; DataHub numbers are a reference from a second machine.

3. RESULTS

3.1. Stage benchmarks

Table 1 shows that on CPU the cost is almost all prefill: about 97% of grounding time and 90–97% of planner time. The cost therefore grows with the number of image tokens and context tokens, not with the length of the output. On the L4 each stage is 17–37× faster, and the prefill share drops to 33–70%, so decode and fixed overheads start to matter. The pixel budget is a clear trade-off. On the L4, going from 125k to 500k pixels costs 2.1× the latency and 2.0× the energy, and raises accuracy from 0.46 to 0.74. CPU accuracy follows the same trend. Accuracy on the eval split (0.71/0.56) is close to dev (0.74/0.57), so our dev results are not tuned to the dev set. As an extra reference point, UI-TARS-7B (Q4_K_S) reaches 0.90 at 0.66 s on the L4.

3.2. Pipeline placements

Table 2 compares the placements on the same AWS machine. B3 is the fastest at 1.11 s per step, about 50× faster than B1. B2 is only 1.75× faster than B1, because once grounding moves to the GPU, the CPU planner takes 98% of the step. B2 also wastes energy. The GPU idles at about 27 W while it waits for the planner, so roughly 94% of B2's 920 J per step (27 W × 32 s) is idle power, and this does not even include CPU energy. The same CPU work is about 1.7× slower on the AWS machine (4 physical cores) than on DataHub, which is why we compare devices only on the same machine.

3.3. Issues found during measurement

Several problems would have given wrong results if we had not caught them. Prompt caching made repeated runs look 27× faster, so it is now off by default. The parser ignored box-shaped outputs, so we now report strict and lenient accu-

Stage benchmark	CPU p50 (s)	L4 p50 (s)	Speedup	L4 TTFT share	L4 energy (J/req)	L4 acc. (lenient / strict)
Grounding, 125k px	4.4	0.23	19×	43%	18.7	0.46 / 0.43
Grounding, 250k px	7.9	0.30	27×	55%	24.3	0.58 / 0.49
Grounding, 500k px	15.6	0.47	33×	70%	36.5	0.74 / 0.57
Grounding, 500k px, eval split (891)	–	0.47	–	70%	37.1	0.71 / 0.56
Planner, 500-token context	4.8	0.28	17×	33%	22.0	–
Planner, 2,000-token context	18.2	0.69	26×	48%	46.1	–
Planner, 4,000-token context	39.5	1.07	37×	65%	69.8	–

Table 1: Stage benchmarks on the dev split unless noted. CPU column is DataHub (8 threads), where prefill is about 97% of grounding time and 90–97% of planner time. CPU grounding accuracy at 125k/250k/500k is 0.45/0.54/0.75 (lenient). Pass-to-pass standard deviation on the L4 is below 0.01 s.

Placement	Step p50 (s)	Planner (s)	Grounding (s)	GPU energy (J/step)	GPU memory (GB)
B1: all CPU (AWS)	56.1	31.6	24.8	1,570*	0
B2: grounding GPU, planner CPU (AWS)	32.0	31.6	0.47	920	3.9
B3: all GPU (AWS)	1.11	0.55	0.48	74	6.3
B1: all CPU (DataHub, reference)	33.7	18.1	15.4	–	–

Table 2: One pipeline step (planner with 2,000-token context, grounding at 500k px; preprocessing is about 0.1 s). *GPU is unused in B1, so this is idle GPU power only. Pipeline accuracy on 20 steps is 0.65–0.70 (lenient) for all placements.

racy separately. llama.cpp split the context across 4 slots, so we now use one slot. Using the wrong coordinate format for Qwen2.5-VL (`norm1000` instead of `pixel`) dropped accuracy from 0.74 to 0.20. The default script listed different models from the ones we actually ran, so we now check model hashes and log every setting. Finally, a CUDA build can still use the GPU with `-ngl 0`, so CPU mode hides the GPU, and we confirm 0 MB of GPU memory in every CPU run.

3.4. Limitations

EC2 does not expose CPU power, so all energy numbers are GPU only. DataHub and AWS use different llama.cpp builds. Planner inputs are synthetic, pipeline grounding uses ground-truth instructions, and pipeline accuracy is based on 20 steps. We did not have access to an NPU.

4. PHASE 2 DIRECTION

On the L4, B3 is the fastest placement for every input we tested, so choosing a different fixed placement will not improve latency. What our data does show is that cost and accuracy depend strongly on the input. The pixel budget alone changes energy by 2× and accuracy by 0.28, and 46% of targets are already found at 125k pixels. In Phase 2 we will build an input-adaptive policy that picks the pixel budget, and the placement when there is an energy or latency limit, for each step. For example, the policy can start at a low budget and move to a higher one only when a cheap check suggests the prediction is unreliable. We will compare it with the best fixed setting (B3 at a fixed pixel budget) and aim for the same accuracy at lower latency and energy, or higher accuracy at the same cost, with one final test on the eval split. The B2 result also shows that any setting where one device waits for the other should count idle power. If time allows, we will also reuse vision encoder work across consecutive screenshots, which are often mostly unchanged. We will confirm the exact definition of "best fixed placement" with the instructor.

Code and data. `https://github.com/lwangjt/cse291a-ecpipe` contains the harness, configs, dataset preparation with the pinned split, all raw results and logs. `PHASE1.md` lists the exact commands, model downloads and hashes.

APPENDIX A. TIMELINE AND MILESTONES

Mid–late Oct	Built the `ecpipe` harness and smoke tests.
Oct 29–31	Pinned data split and models. DataHub CPU runs: grounding, pixel sweep, planner sweep, B1. Fixed caching, slot and box-parsing issues.
Nov 1	Set up AWS L4 instance and CUDA build of llama.cpp.
Nov 1–2	GPU stage runs, same-machine B1/B2/B3, eval check. Fixed coordinate-format and model-mismatch issues.
Nov 3–4	Phase 1 report and code release.
Phase 2, wk 1	Confirm objective; per-sample analysis by pixel budget; fit a latency/energy cost model.
Wk 2	Design and implement the adaptive policy in `ecpipe/pipeline.py`.
Wk 3	Evaluate on dev against fixed baselines (B3 at 125k/250k/500k).
Wk 4	Stretch: vision encoder reuse across steps.
Wk 5	Final eval-split test, final report and code.

REFERENCES

[1] K. Cheng et al., "SeeClick: Harnessing GUI grounding for advanced visual GUI agents," ACL, 2024 (ScreenSpot benchmark).

[2] G. Gerganov et al., llama.cpp, `https://github.com/ggml-org/llama.cpp`.

[3] Qwen Team, "Qwen2.5-VL technical report," arXiv:2502.13923, 2025.